

Assignment 3

TA: Qibing Bai

Due Date: December 15, 11:59 pm

Total points available: 100 pts.

Note: Please note that external references are totally allowed only if you give appropriate reference. There is no required format of reference. Please elaborate on your answers as well. (not just give a number etc)

Problem 1: DDQN for a Mario-Playing Agent [50 pts.]

In this problem, you will implement the Double Deep Q-Network (DDQN) algorithm to train an AI agent that can play Super Mario Bros using the OpenAI Gym environment. The [code skeleton](#) is provided in the Jupyter notebook file. You must complete the following three components of the DDQN algorithm:

1. Epsilon-Greedy Action Selection (*act* method): Implement the exploration-exploitation trade-off that balances random exploration with learned policy exploitation.
2. Convolutional Neural Network Architecture (*_build_cnn* method): Build a deep CNN that processes 84×84 grayscale frame stacks to output Q-values for each action.
3. TD Estimate and TD Target Calculations (*td_estimate* and *td_target* methods): Implement the core DDQN algorithm where the online network selects actions and the target network evaluates them, following the Double Q-learning principle.

After finishing the components, please train the agent and submit the IPYNB file containing your implementation and results.

Problem 2: SAC for Continuous Control [50 pts.]

In this problem, you will implement the Soft Actor-Critic (SAC) algorithm to train an AI agent for the Pendulum-v1 continuous control task using the Gymnasium environment. SAC is a state-of-the-art off-policy algorithm that maximizes both expected reward and policy entropy for improved exploration and robustness. The [code skeleton](#) is provided in the Jupyter notebook file. You must complete the following two components of the SAC algorithm:

1. Automatic Entropy Tuning (*update_model* method - TODO 1): Implement the dual optimization problem for automatic temperature parameter adjustment. This mechanism adaptively controls the trade-off between exploration (entropy maximization) and exploitation (reward maximization) by ensuring the policy entropy matches a target value.
2. Actor Loss with Entropy Regularization (*update_model* method - TODO 2): Implement the maximum entropy policy gradient that trains the actor to maximize Q-values while maintaining high policy entropy. This is the core of SAC's exploration strategy, where the actor balances improving performance with exploring diverse behaviors.

After finishing the components, please train the agent and submit the IPYNB file containing your implementation, training plots, and results.